

File Transfer API

v. 1.0

Version Control

Version	Release Date	Author	Comments
1.0	29.09.2021	Giovanni Mele	First version
1.1	12.10.2021	Giovanni Mele	Review after comments
1.2	17.03.2022	Jamal Abdelkhalek	Review after implementation and first tests

Contents

Contents.....	3
1. Context	4
2. File Transfer API	4
2.1. File Transfer from Equipment to Cloud	5
2.2. File Transfer from Cloud to Equipment	8
2.3. REST API	10
2.3.1. Register an Edge File Transfer service	11
2.3.2. Receive a file	11
2.3.3. Receive a file transfer status	12
2.3.4. Keepalive API	13
3. Edge-Backend architecture.....	14
4. Open points	16

1. Context

This document contains the specification for the API used to transfer files between the equipment and the Edge device the two directions. The API is agnostic and can be used by any service in the cloud to exchange PDFs, images, long XMLs, or any file that can be consumed or produced by devices in the equipment.

2. File Transfer API

The file transfer API describes how the file will be transferred between a dedicated equipment service and the Edge device through an edge module. As the equipment cannot access the cloud directly, transferring files through the edge device is mandatory.

The HTTP protocol version 1.x has been chosen to transfer files as it natively supports big file transfer in multipart payload. For this reason and because the transfer occurs locally into the equipment (same network), we will not implement any resumable custom protocol. The FTP (or SFTP) protocol has been discarded as we want to be able to transfer meta data with the file.

The securing of this protocol will be done using HTTPS protocol. The first version will use simple HTTP as the equipment does not manage X.509 certificates yet. The HTTPS protocol will be introduced with the “Unified data and process Engine Module” as described in document [MachineEvolutivity](#).

In addition to the file transfer, the protocol must also provide interfaces to return the status of the transfer to the call initiator.

Transferring files from or to the equipment must also support multiple producers and consumers on both sides. We can easily imagine Bobst Connect wants to transfer a recipe composed by several ‘files’, each one targeted for a service running on the equipment (JRM, iQ 500). The equipment may also generate files of several types, like quality reports or information about a rejected item from iQ 500, production status (JRM), etc...

To support multiple sources, we propose to tag transferred files with a type of information (application type), letting the consumer subscribe to the resources it needs.

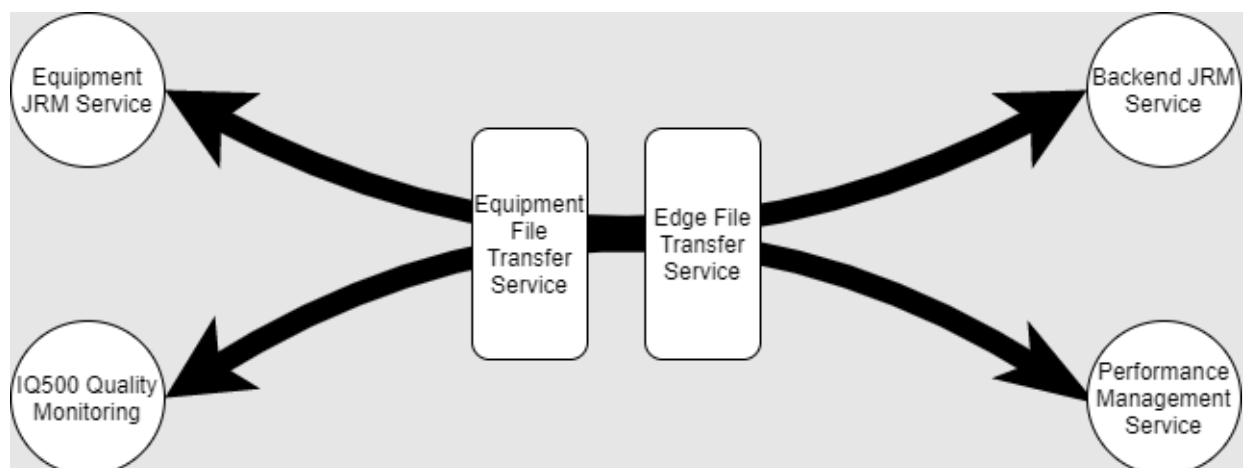


Figure 1 File transfer producer/consumer

File transferred on the equipment must contain all information (like job description) to be processed by the equipment as the Bobst Connect backend cannot send meta-data to the Edge device due to limited communication possibilities offered by Azure (no messaging from IOT Backend to IOT Edge devices). On the other hand, when the equipment sends a file to be processed by the Bobst Connect backend, it must give enough information to let the backend process the file without having to open it (due to the high number of equipment that may send files to the backend).



File transfer is a way to transfer massive amounts of information between the equipment and the Bobst Connect, like iQ 500 pdf files, high-definition images, etc... For small files, we should consider using dedicated interfaces and use IOT Messages to send data from the equipment.

2.1. File Transfer from Equipment to Cloud

The equipment must be able to send files on the backend. The file may be consumed by one or several services on the backend. Once the file has been effectively transferred to the appropriate backend storage location, the Edge device will notify the backend with an IOT Message containing all the metadata transmitted by the equipment.

The file metadata

```
{
  "fileName" : string,    // The file name
  "timestamp" : string,   // with format yyyy-mm-ddTHH:MM:ss.fffZ
  "applicationType" : string, // It is the file tag used to target a remote service
  "jobId" : string,
  "jobName" : string,
  "jobRef" : string,
  "recipeId" : string,
  "recipeName" : string,
  "recipeNumber" : string,
  "data": {
    key1 : value1,
    key2 : value2,
    ...
  }
}
```

Figure 2 File Metadata JSON

Field	Description	Mandatory
fileName	The name of the transmitted file. Should be unique on a given equipment for a given type (do not send always the same file named Report.pdf).	Yes
timestamp	Tells when the file was generated, in UTC. The timestamp is a string with format "yyyy-mm-ddTHH:MM:ss.fffZ"	Yes

applicationType	Describe the application type, i.e QCS, JRM, DFE, etc... This field is used to route the file to the correct equipment or backend service. This is an enum managed by IOT team. It will be enhanced after discussion with the different team needing to transfer files....	Yes
jobId	The job id (or operation id) linked to the file, if any. Can be null.	No
jobRef	The job reference, usually coming from an external MSO	No
jobName	The job name, if any.	No
recipeId	The recipe id, if any.	No
recipeName	The recipe name, if any.	No
recipeNumber	The recipe number if any.	No
data	Additional data, if any, that may contain useful information for the backend service, i.e, fault code, fault location, etc. It is a dictionary key = value.	No

Table 1 File metadata definition

We consider that usually, a generated file will be linked to a job on the equipment. For this reason, the job/recipe information is part of the file metadata. The type field is used to give the file purpose and target the right consumer on Bobst Connect backend. If a file is not linked to a job/recipe, these fields can be not present.

To support a variety of file types, a dictionary field is part of the metadata. It may contain elements for fault detection (fault code, location, etc....) or any useful data extractable by the backend.

Once the file has been effectively pushed on the Bobst Connect backend, or in case of error/timeout, the Edge device will notify the equipment of the transfer status to let the sending service decide of the action to take.

The transfer status:

```
{
  "fileName" : string, // The file name
  "applicationType" : string, // It is the file tag
  "timestamp": string, // with format yyyy-mm-ddTHH:MM:ss.fffZ
  "status": string, // a string that should represent a FileTransferStatus code
  "message": string
}
```

Figure 3 Transfer status JSON

Field	Description	Mandatory
fileName	The name of the transmitted file.	Yes

applicationType	Describe the application type, i.e QCS, JRM, DFE, etc... This field is used to route the file to the correct equipment or backend service. This is an enum managed by IOT team. It will be enhanced after discussion with the different team needing to transfer files....	Yes
timestamp	Tells when the file was generated, in UTC. The timestamp is a string with format "yyyy-mm-ddTHH:MM:ss.fffZ"	Yes
status	Code describing if the file was successfully sent or not. Should be one of the enum FileTransferStatusCode	Yes
message	A message describing the transfer status. Can be empty.	No

Table 2 File transfer status definition

The edge device will also notify the Bobst Connect backend that a file has been sent, encapsulating the file metadata in a **MachineEvent** message of type 'fileReceivedFromEquipment'.

```
{
  "type": " fileReceivedFromEquipment",
  "equipmentId": string,
  "timestamp": string, // with format yyyy-mm-ddTHH:MM:ss.fffZ
  "datasource": string,
  "fileName": string, // The file name
  "applicationType": string, // It is the file tag used to target a remote service
  "jobId": string,
  "jobName": string,
  "jobRef": string,
  "recipeId": string,
  "recipeName": string,
  "recipeNumber": string,
  "data": {
    key1 : value1,
    key2 : value2,
    ...
  }
}
```

Figure 4 MachineEvent for file upload

The new message field definitions are described in Table 1.

Use case: *quality report at the end of a job*

In this example, we describe how a QCS report can be used by several services the Bobst Connect backend.

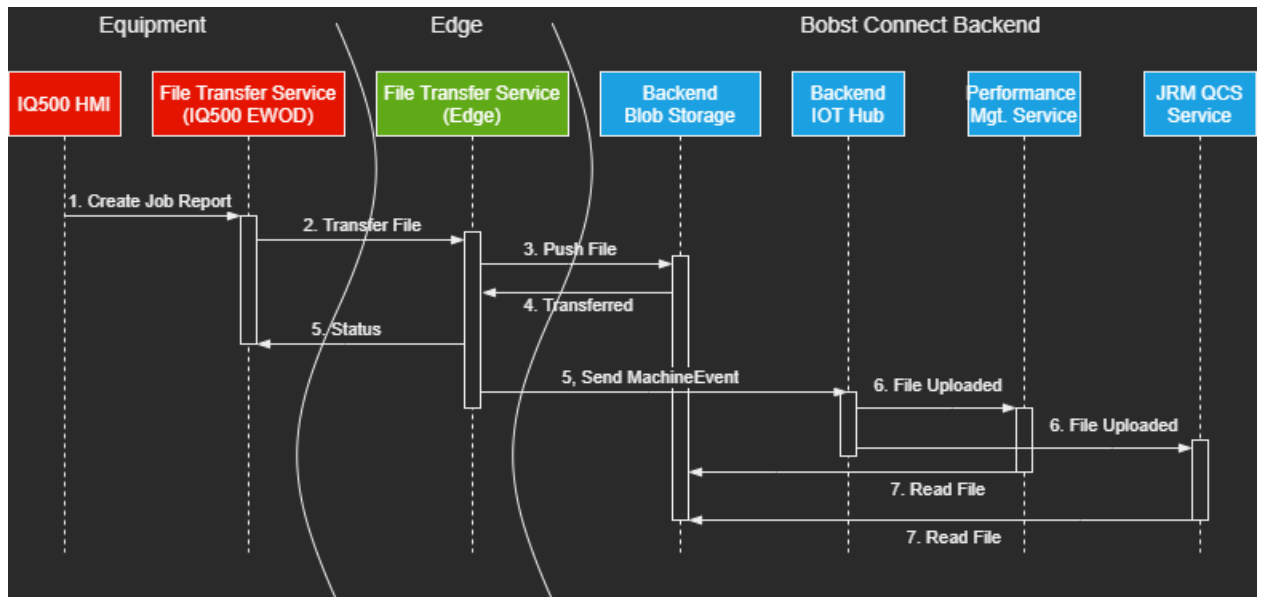


Figure 5 Sending QCS report to Bobst Connect

1. The iQ 500 HMI generates a QCS report at the end of an active job and pushes it on a shared location for transfer
2. The iQ 500 Ewod responsible for the File Transfer Service will pick up the file and send it to the Edge device File Transfer service (module), giving all file metadata.
3. The Edge module transfers the file on an Azure Blob container in the Bobst Connect backend
4. The Edge device waits for confirmation that the file has been sent on the backend
5. Once the confirmation is received (it can be after hours if the connection from the equipment plant to the internet is very slow!), the Edge module:
 - a. Sends a confirmation status to the Ewod File Transfer Service
 - b. Sends a Machine Event to the Bobst Connect backend with the file metadata and the equipment id
6.
 - a. The Performance Management service is notified a QCS file has been received and stores it in a local storage so it can be used to display a job quality result
 - b. The JRM QCS service is notified a QCS file has been received and stores it in a local storage so it can be used in the JRM QCS dashboard (example!)
7. The backend services read the received file and eventually store it in the service storage for future use



In the future, we expect to receive the QCS file information in the Machine Event endJob event directly. Today, it is not the case.

2.2. File Transfer from Cloud to Equipment

The Bobst Connect backend should also be able to send files to the equipment. Files should be used to transfer huge and complex information/configuration to the equipment. The file must contain all the information needed by the equipment service using the file.

To transfer a file to equipment, the Bobst Connect backend must use equipment dedicated blob storage container. The container will contain a folder per supported applicationType.

The edge device will be triggered by a twin configuration update to download the file in the cloud. When the Edge device detects a new file to download path in the twin, it will download it onto the device and once finished, push it on the File Transfer Service of the equipment. It will fill the file metadata structure as described in Figure 2. All information about the job will remain empty as the additional data. Only the fields *fileName*, *applicationType* and *timestamp* will be filled.

On the equipment side, once the file is received and processed, the edge device is expecting to receive a file transfer status as described in Figure 3. This information will be sent to the backend using a new **MachineEvent** message of type 'fileSentToEquipmentStatus':

```
{
  "type": "fileSentToEquipmentStatus",
  "equipmentId" : string,
  "timestamp": string,    // with format yyyy-mm-ddTHH:MM:ss.fffZ
  "datasource" : string,
  "fileName" : string,    // The file name
  "applicationType": string, // It is the tag used to target a remote service
  "status": string, // a string representing a FileTransferStatusCode
  "message": string
}
```

Figure 6 MachineEvent for file download status

The new message field definitions are described in Table 2.

Use case: *Job quality check configuration for iQ 500*

In this example, we describe how a QCS quality check configuration can be sent on the equipment's iQ 500.

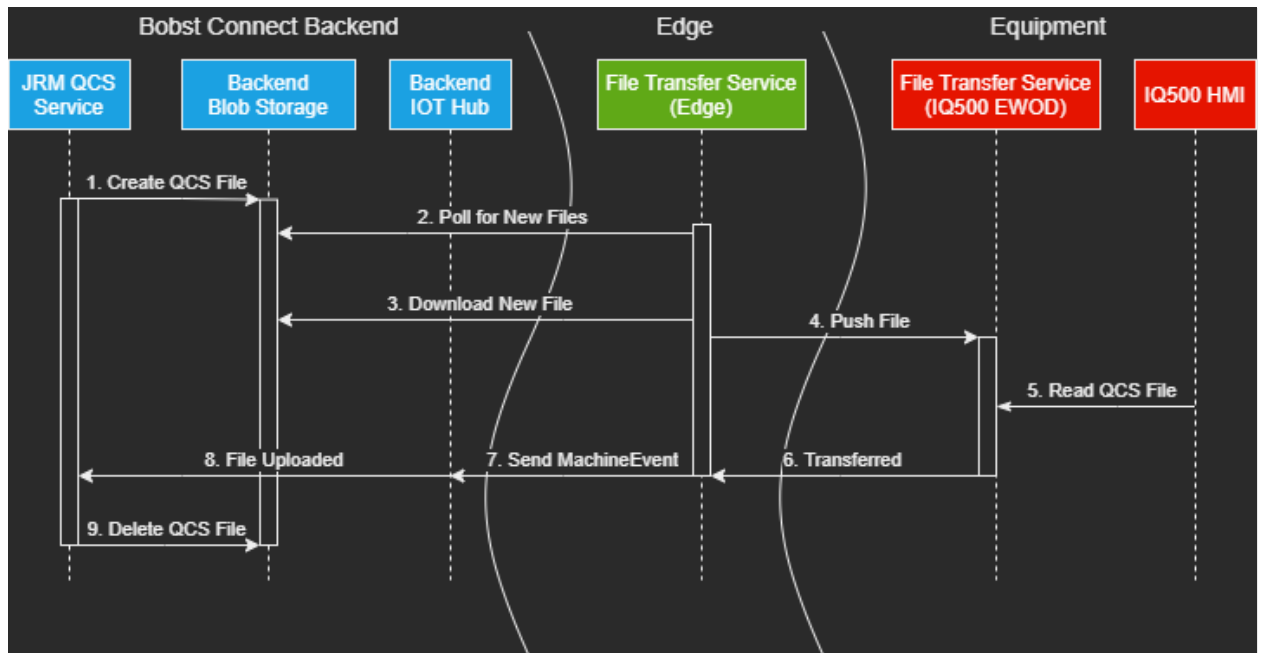


Figure 7 Sending QCS configuration to iQ 500

1. The backend JRM QCS Service creates a QCS configuration file for a given job and equipment (<container=machineId>/toEquipment/qcs/<filename.pdf>)
2. The device management service will update the twin of the edge device. By updating the value of the storage account name, generating a new sas token, and inserting a new file to download path.
3. The Edge File Transfer Service detects a new configuration in which there is a new fileToDownload path. It downloads it.
4. Once downloaded locally, the Edge File Transfer Service sends the file to the equipment's File Transfer Service
5. The equipment's iQ 500 HMI reads the file
6. The equipment's File Transfer Service notify the Edge File Transfer Service of the file transfer status
7. The Edge File Transfer Service sends a MachineEvent message to the Azure IOT Hub with the file transfer status
8. The Bobst Connect JWM QCS Service is notified of the file transfer status
9. The Bobst Connect JWM QCS Service deletes the transferred file

2.3. REST API

Both services running on the equipment and on the Edge will implement the same interface to receive file and send a transfer status.

On the Equipment side, the service will have to handle an additional REST API to allow the registration of the Edge module on the equipment's File Transfer Service and avoid having to configuration in the equipment. The same principle is used with the Machine Event service.

2.3.1. Register an Edge File Transfer service

This REST API is only available on the equipment's File Transfer Service. It is used by the Edge device to register a callback URI that will be used by the equipment's File transfer Service to send file transfer status or to send file for the Bobst Connect backend.

The equipment service callback registration interface is:

GET /filetransfer/registercallback&callbackuri={callbackUri}

Query parameters:

- **callbackuri**: this is the HTTP callback URI where to send transfer request or status on the Edge device

Return codes:

- **204 OK**: if the registration was done successfully
- **4xx or 5xx**: if the registration failed

2.3.2. Receive a file

This REST API is meant to receive file with the HTTP protocol. It must handle at least 4GB file size. The content type of the request must be **multipart/mixed** with 2 sections:

Section name	Content	Description
file	Content-Disposition: form-data Content-type: application/octet-stream name: 'file' filename : the file name transferred	Contains the file to transfer. The file should be streamed and not sent as plain text data.
metadata	Content-Disposition: form-data Content-type: application/json name: 'metadata'	Contains the json describing the transferred file as described in Table 1 File metadata definition

Table 3 File transfer HTTP multipart description

The file transfer interface (for both the Edge and the equipment services) is:

POST /filetransfer

We will not describe the multipart/mixed boundaries as it has been done just before....

Return codes:

- **204 OK**: if file was successfully received
- **4xx or 5xx**: if the file transfer failed

Request content type : **application/octet-stream**

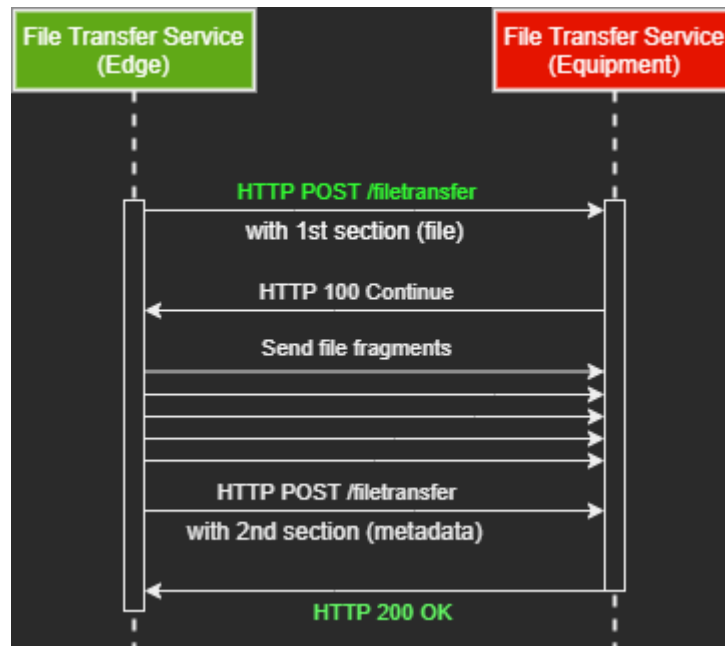


Figure 8 HTTP request for file transfer

The HTTP request in green are issued by the File Transfer Services. The HTTP request between the POST and the 200 OK is managed by the HTTP protocol itself and doesn't need any action by the developer.

The Edge File Transfer Service, once the file is effectively transferred on the remote Azure Blob container, will send a MachineEvent message as described in Figure 4.

How to simulate a client sending a file with curl command:

```
curl -i -X POST -H "Content-Type: multipart/mixed" -v -F file=@testfile.log -F "metadata={\"fileName\": \"qcsConfigRecipeToto.pdf\", \"timestamp\": \"2021-10-03T04:28:00.000Z\", \"applicationType\": \"qcs\", \"source\": \"test\"};type=application/json" http://172.16.6.1:8080/filetransfer
```

2.3.3. Receive a file transfer status

Once we have sent a file to a remote File Transfer Service (Edge or equipment), we need to know if the transferred file was effectively handled by the service. We cannot rely only on the HTTP 200 OK we receive when the POST request is done to send the file as it only says the file was received, but not if it was effectively processed.

«Effectively processed» has different meanings depending on the service location:

Service location	Description
Edge	When the Edge File Transfer Service sends the status, it only means that the file was transferred on the remote Azure Blob container. We do not know if it was processed by the backend services

Equipment	When the equipment File Transfer Service sends the status, it means the equipment service using the transferred file was able to use it. Once the Edge File Transfer Service receives the status, it will notify the backend the file was correctly processed using a MachineEvent as described in Figure 6
------------------	---

Table 4 File transfer status description

To let the File Transfer Service, identify the file concerned by the status received, the status request body will contain a json as described in Figure 3 and Table 2.

The file status interface is:

POST /filetransfer/status

The POST payload is a json as described just before.

Return codes:

- **200 OK:** if the status was being handled.
- **404:** if the status is for an unknown file

Request content type : **application/json**

2.3.4. Keepalive API

The keepalive or ping API can be used to verify the File Transfer Service is still available.

The keepalive (ping) interface is:

GET /filetransfer/ping

In case of success, the ping answer will return the service UTC time when the ping request was received

Return codes:

- **200 OK** : if the service is ready to send/receive. The message payload will contain an UTC timestamp. This timestamp represents the time when the ping request was received.

For example:

2021-08-23T14:56:23.123Z

- **4xx or 5xx:** if the registration failed

Response content type : **text/plain**

3. Edge-Backend architecture

In this chapter, we will propose the principle of an architecture needed on the Edge device and the Bobst Connect backend to fulfill the requirements for transferring file from and to the equipment.

The main idea is to transfer files using a container per equipment on dedicated Blob storage on the Azure backend.

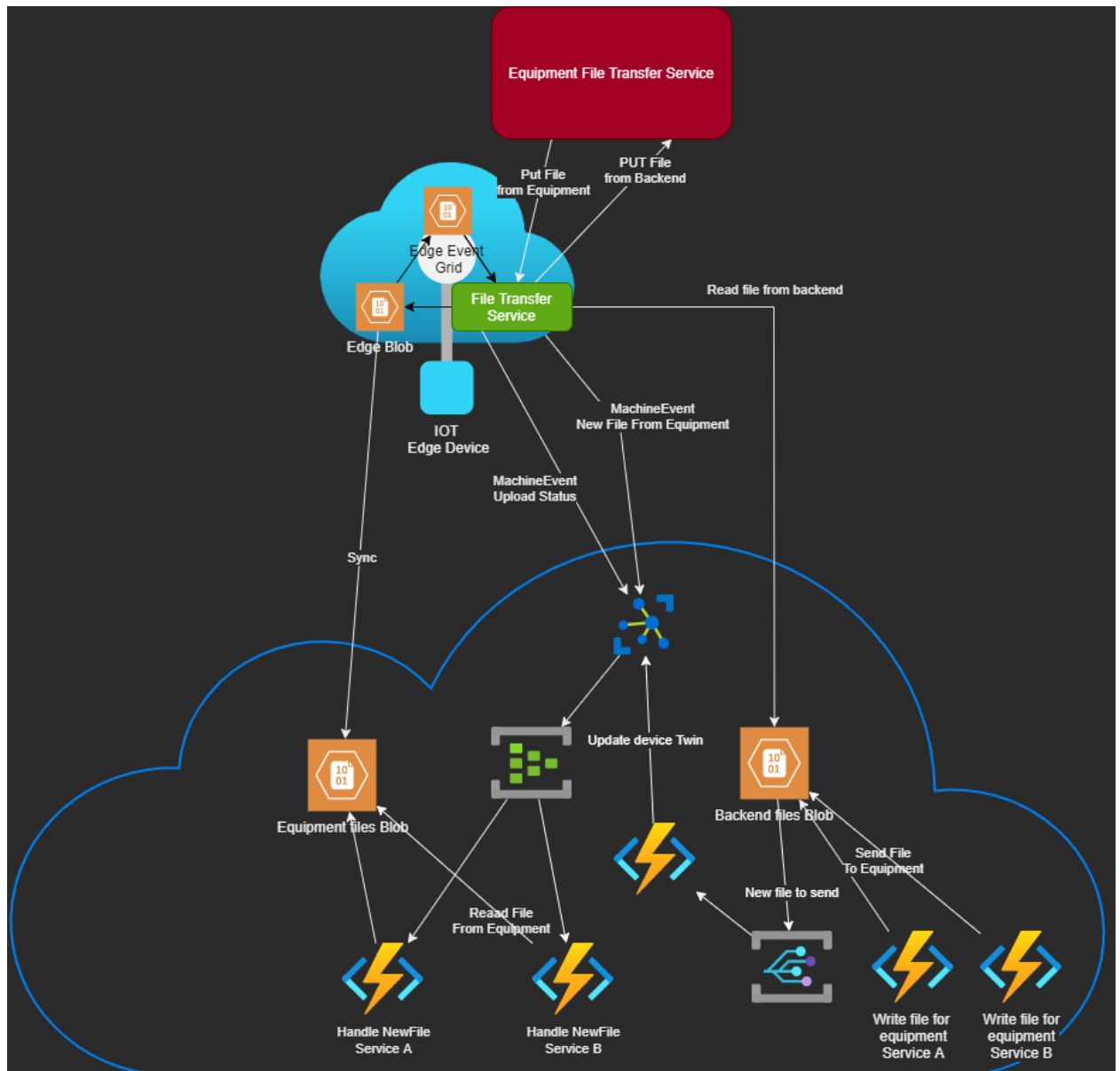


Figure 9 Edge-Bobst Connect backend architecture

3.1. Edge device

On the edge device, a new module FileTransferService must be implemented. It will offer the REST API as described in §2.3.

Transferring file to the backend will use an Edge IOT Blob, mapped to a remote container on the Azure backend. Once the file is effectively transferred to the remote container, it must be deleted from the local IOT Edge Blob container.

As the FileTransferService module must notify the equipment's File Transfer Service (using an HTTP REST API) and send a Machine Event to notify the Bobst Connect backend, an IOT Edge Event Grid module must be instantiated and listen to Blob file deletion event and forward them to the module.

On the Edge device, we will have 3 modules:

- FileTransferService (new module)
- IOT Edge Blob (to transfer file)
- IOT Edge Event Grid (to notify modules the file has been added/deleted)

Sending files from the backend to the equipment will also use a Blob container, but as we cannot automatically transfer new files on a Blob container running on the Edge, the FileTransferService module will be notified of a new file via the device twin. As soon a new file is written on the container and the twin updated with the file path of the file to send, it will be downloaded on the Edge device and forwarded to the equipment using its REST API.

When the equipment's File Transfer Service confirms the file is received, the FileTransferService module generates a Machine Event to the Bobst Connect backend. It will be under the responsibility of the service sending the file to delete it.

As the edge File Transfer Service must be capable to send files on different peripheral, part of an equipment, the module must support several data sources (the equipment peripheral remote ip address/port), each data source being linked to one or several file types.



The equipment must access its own container using a SAS token refreshed periodically. It must not be able to retrieve data from another equipment.



IOT Edge Event Grid is still in preview (according Azure documentation. Clarifications have been asked to Microsoft to understand if this is still the case and what would be the GA of this product.

3.2. Bobst Connect Backend

The Bobst Connect backend must provide 2 different Blob storages used to send and receive files.

When sending files to the equipment, the backend will first push the file on the dedicated Blob storage which will trigger an update of the device TWIN to notify the remote Edge module. Once the file has been transmitted and accepted by the equipment, the producer service must purge the sent file upon reception of the file transfer status received in a dedicated Machine Event. A purge mechanism must also be put in place to do file cleanup.

When a file is received, all backend services interested in that file (application type) will be notified by receiving a dedicated Machine Event. The services are responsible for storing the received file elsewhere for their own usage. The container must be periodically cleaned to delete received files.

All notifications coming from the Edge File Transfer Service will be encapsulated in MachineEvent events (§Figure 4 and §Figure 6)Figure 4 MachineEvent for file upload. Services interested in these events must subscribe to the appropriate name space in the IoT Event Hub. The connected service will be responsible for handling the events as they want.

The backend will also periodically refresh the equipment container's SAS token for security reasons, as it is done already for configuration upload.

4. Open points

The current file transfer protocol assumes that the file producer pushes the file on the remote system.

Do we want to support the following use cases:

1. The equipment requests a file on the Bobst Connect backend?
2. The Bobst Connect backend requests a file on the equipment?

In my opinion, we shouldn't implement these use cases as the goal of the connection of the equipment to Bobst Connect is also to modernize the interfaces provided by the different services. Allowing such use cases is a major brake on innovation.